Ejemplo en java

```
public class CodigoIntermedio {
    public static int ejecutar() {
        int a = 5;
        int b = 10;
        int t1 = a + b;
        int t2 = a + b;
        int x;
        if (t1 > 10) {
            x = 1;
        } else {
            x = 0;
        }
        int y = x;
        int z = 20 + 30;
        int w = 100;
        return y;
        // Código inalcanzable:
        // w = 200;
    }
    public static void main(String[] args) {
        System.out.println(ejecutar());
    }
}
```

Observaciones de optimización

Este código contiene varias oportunidades de optimización:

1. **Subexpresión común**

   int t1 = a + b;

   int t2 = a + b;

   La suma a + b se calcula dos veces. Puede reutilizarse:

   int t1 = a + b;

   int t2 = t1;

2. **Propagación de constantes**

   int a = 5;

   int b = 10;

int t1 = a + b;

Se convierte en:

int t1 = 15;

3. **Plegado de constantes**

   int z = 20 + 30;

   Se simplifica a:

   int z = 50;

4. **Código muerto**

   int t2 = ...

   int z = ...

   int w = 100;

   Ninguna de estas variables se utiliza antes del return, por lo que pueden eliminarse.

5. **Código inalcanzable**

   return y;

   w = 200;

   La asignación w = 200 nunca se ejecutará porque está después del return.

Versión optimizada

```
public class CodigoOptimizado {
    public static int ejecutar() {
        int t1 = 15;
        int x;
        if (t1 > 10) {
            x = 1;
        } else {
            x = 0;
        }
```

```
        return x;
    }
    public static void main(String[] args) {
        System.out.println(ejecutar());
    }
}
```

En esta versión se aplican: **eliminación de sub expresiones comunes, propagación de constantes, plegado de constantes, eliminación de código muerto y eliminación de código inalcanzable**.

Para este código de tres direcciones:

1: a = 5

2: b = 10

3: t1 = a + b

4: t2 = a + b

5: if t1 > 10 goto L1

6: x = 0

7: goto L2

L1:

8: x = 1

L2:

9: y = x

10: z = 20 + 30

11: w = 100

12: return y

13: w = 200

Paso 1. Identificar los bloques básicos

B1

1: a = 5

2: b = 10

3: t1 = a + b

4: t2 = a + b

5: if t1 > 10 goto L1

B2

6: x = 0

7: goto L2

B3 (L1)

8: x = 1

B4 (L2)

9: y = x

10: z = 20 + 30

11: w = 100

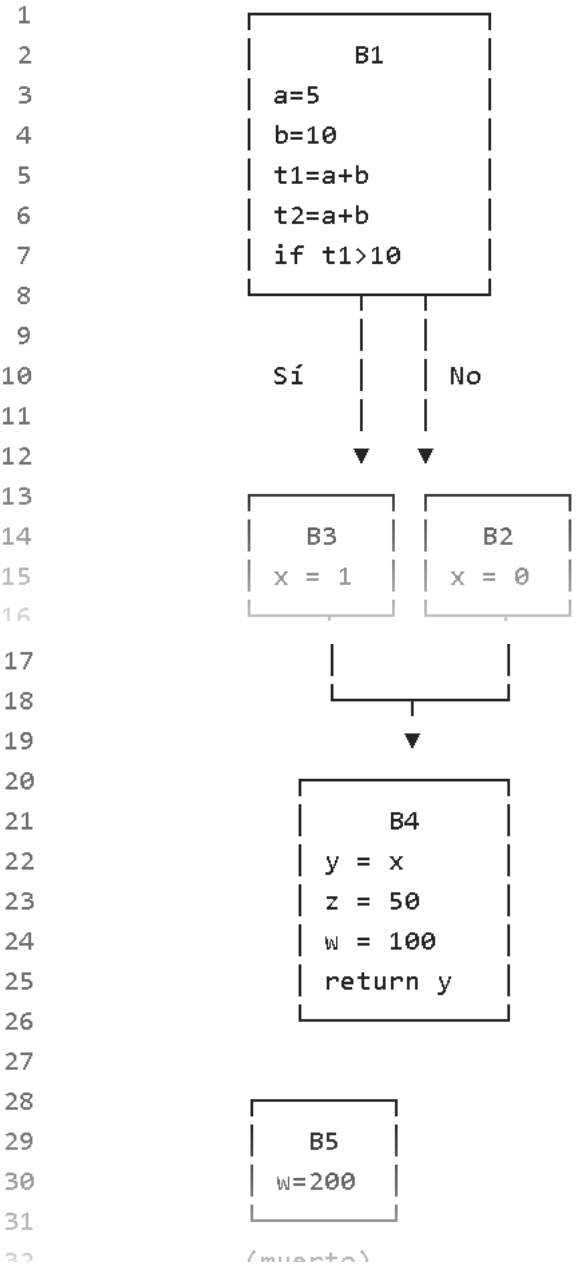
12: return y

B5

13: w = 200

B5 es un bloque inalcanzable porque está después del return.

Paso 2. Construir el CFG



CFG en forma simplificada

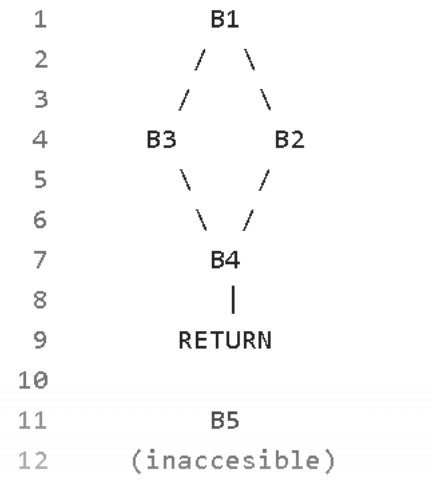


Tabla de sucesores

Bloque	Sucesores
B1	B2, B3
B2	B4
B3	B4
B4	Fin
B5	Ninguno (inalcanzable)

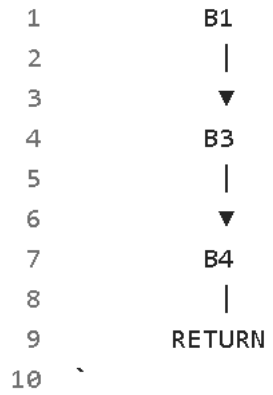
Grafo después de la optimización

Como a=5 y b=10, entonces:

t1 = 15

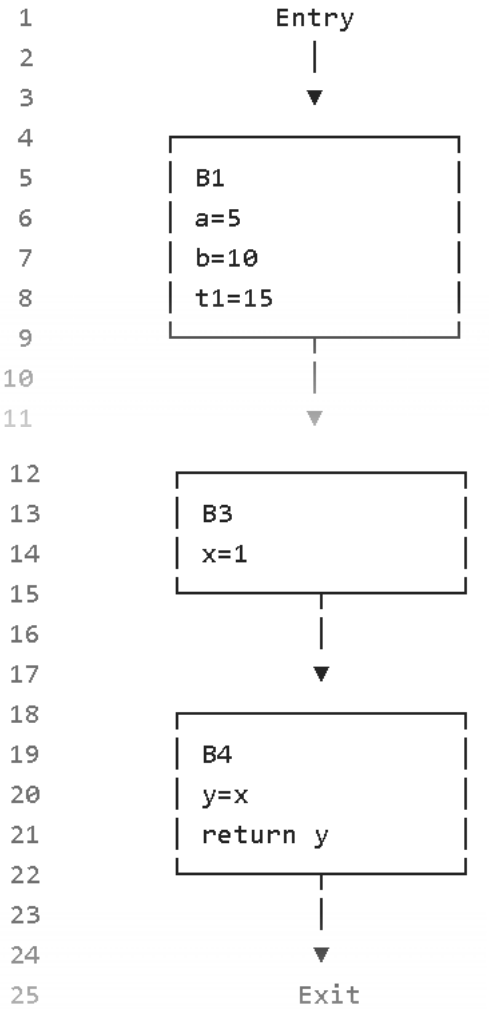
if (15 > 10)

La condición siempre es verdadera, por lo que el compilador puede eliminar la rama falsa.



Este CFG optimizado muestra la aplicación de **propagación de constantes**, **evaluación de condiciones constantes** y **eliminación de código muerto**.

CFG Optimizado



Árbol de optimizaciones aplicadas

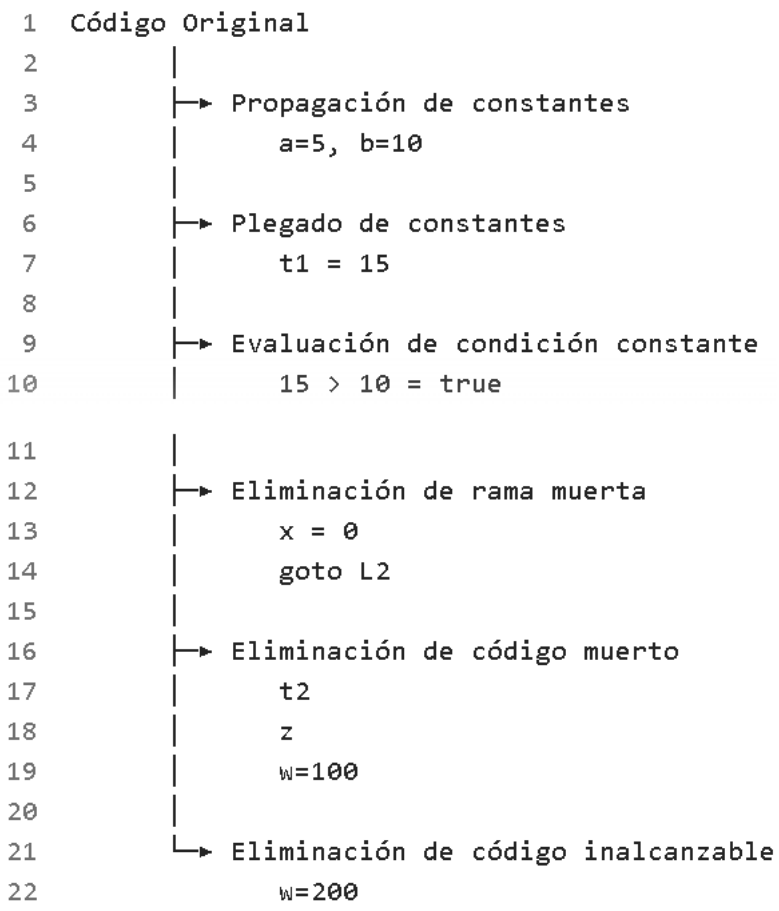


Tabla de transformaciones aplicadas

Elemento	CFG Original	CFG Optimizado	Optimización
Rama falsa B2	Sí	No	Eliminación de código muerto
Condición	t1 > 10	15 > 10 = true	Propagación y plegado de constantes
Nodo B5	Sí	No	Eliminación de código inalcanzable
Número de caminos	2	1	Simplificación del flujo
Complejidad ciclomática	2	1	Reducción de decisiones